Qwen3-Coder-Next Tier 3

Part 7 Detailed Implementation Specification

Evaluation Layer

Principal engineering specification for the judge, rubric, and approval gate that sits after review and before deployment.

Part	7
Name	Evaluation Layer
Primary role	Determine whether a completed change actually satisfies the task, not just whether it compiles or passes tests.
Depends on	Part 3 Planning, Part 4 Research, Part 5 Memory, Part 6 Testing & Review

Key Insight

Testing proves that code behaves as written.
Evaluation proves that the code solves the intended problem.
This layer is the semantic gate that prevents technically correct nonsense from reaching deployment.

1. Purpose

The Evaluation Layer exists to decide whether a proposed change is actually correct relative to the original task, not merely whether it compiles, passes tests, or looks plausible in review. It is the semantic checkpoint that sits between the Testing/Review stages and deployment.

This part solves the core failure mode of autonomous coding systems: a solution can satisfy local implementation checks while missing the user's real intent. The evaluator compares the finished work against the task specification, the plan, the evidence produced by testing and review, and any relevant historical memory.

Future parts depend on this layer because every later autonomous loop needs a reliable pass/fail decision and structured feedback. Failure recovery uses the evaluator's rejection reasons. Benchmarking uses its scores. Deployment uses its approval decision. Memory uses its learned failure patterns.

> **Non-goal**
>
> This part does not generate code.
> It does not run tests.
> It does not perform deployment.
> It only judges the result and explains why.

2. Scope

Included	Excluded
Task-spec parsing, rubric construction, evidence aggregation, scoring, decision policy, rejection feedback, audit logging, and integration with LangGraph.	Code generation, code editing, build execution, test execution, deployment orchestration, and direct repository mutation.
Read-only access to plan outputs, test outputs, review notes, memory lookups, diff summaries, and repository metadata.	New architectural components outside the existing architecture. No new agents are introduced here.
Approval / rejection decisions with structured reasons and confidence levels.	Any attempt to replace testing or review with heuristic opinion alone.

The scope is intentionally narrow. The evaluator consumes evidence produced by earlier stages and outputs a decision plus rationale. It is not responsible for creating that evidence.

The evaluator may request additional evidence when required, but it must not silently invent missing facts. Missing evidence is a failure state, not a prompt to guess.

> **Guardrail**
>
> If the evidence is incomplete, the correct action is rejection or escalation, never speculative approval.

3. System Overview

The evaluation flow is a terminal quality gate inside the agent loop. It receives the completed work package and produces one of three outcomes: approve, reject with actionable feedback, or escalate to human review.

```
User / Task
   ↓
```

```
Planner + Research + Coding + Testing + Review
   ↓
Evaluation Coordinator
   ↓
Evidence Collector -> Rubric Builder -> Score Engine -> Decision Policy
   ↓
Approve / Reject / Escalate
   ↓
Deployment or Retry / Recovery
```

The coordinator combines structured inputs from the prior stages: the original task, the implementation plan, the patch summary, test artifacts, review observations, and any relevant memory entries. The score engine then compares the evidence against a rubric that encodes the task's intent.

The decision policy applies thresholds and hard rules. A change that is technically successful but semantically off-target must be rejected. A change that is incomplete but close enough to continue may be rejected with a recovery-oriented reason instead of an immediate dead end.

The output is not just a boolean. It is a machine-readable decision record that can drive recovery, observability, and later benchmark analysis.

4. Internal Architecture

The implementation should be decomposed into small, testable modules with one responsibility each. The core logic should remain deterministic and data-driven.

Component	Responsibility	Depends on	Produces
Evaluation Coordinator	Orchestrates the evaluation pipeline and assembles inputs.	LangGraph state, prior-stage outputs	Evaluation request
Evidence Collector	Normalizes test logs, review notes, diff summaries, and memory lookups into a single bundle.	Filesystem, artifacts, memory reader	Evidence bundle
Rubric Builder	Turns task intent and plan steps into evaluation criteria.	Task spec, plan output, task metadata	Evaluation rubric
Score Engine	Scores each rubric item against evidence.	Rubric, evidence bundle	Per-criterion scores, aggregate score
Decision Policy	Applies thresholds and hard rules to produce approve/reject/escalate.	Scores, confidence, policy settings	Decision record
Feedback Generator	Transforms failures into actionable guidance for the Coding or Recovery Agent.	Rejected rubric items, evidence gaps	Structured feedback
Audit Logger	Persists every decision for traceability and benchmarking.	Decision record, timestamps, task ID	Audit entry

The architecture should follow a one-way flow: assemble evidence, derive rubric, score, decide, publish. Avoid loops inside the evaluator unless the missing information is a bounded retrieval request. Long self-debates are how systems waste time pretending uncertainty is strategy.

```
Evaluation loop: task -> evidence -> rubric -> scores -> decision -> feedback -> audit
```

Design rule

The evaluator must never read its own previous verdict as a source of truth for the same decision cycle.
Each run should be based on fresh evidence and explicit policy.

5. Project Structure

Folder / Module	Purpose
evaluator/	Top-level package for the evaluation layer.
evaluator/coordinator.py	Workflow entry point and orchestration.
evaluator/evidence/	Collectors and normalizers for test, review, diff, and memory artifacts.
evaluator/rubric/	Task-to-criteria translation logic and rubric models.
evaluator/scoring/	Criterion scoring, aggregation, and confidence calculation.
evaluator/policy/	Decision thresholds, hard rules, and escalation logic.
evaluator/feedback/	Rejection explanation and handoff message generation.
evaluator/schemas/	Input/output schemas for requests, scores, and decisions.
evaluator/audit/	Structured logging and persistence helpers.
evaluator/prompts/	Any textual templates used for standardized feedback wording.
tests/evaluator/	Unit, integration, and golden tests for the layer.

The structure separates orchestration from evaluation logic so that scoring and policy can be tested without executing the whole runtime. That separation also makes it easier to replace individual strategies later without destabilizing the contract.

Keep the schemas independent from the coordinator. The coordinator should consume typed inputs and publish typed outputs, not infer structure from ad hoc dictionaries.

6. Data Contracts

All inputs and outputs should be explicit, versioned, and serializable. The evaluator should not depend on implicit ordering or free-form text blobs where structured evidence is available.

Contract	Fields / Shape	Notes
EvaluationRequest	task_id, task_spec, plan_summary, patch_summary, test_artifacts, review_notes, memory_refs, policy_version	Immutable snapshot for one evaluation run
EvidenceBundle	normalized_tests, normalized_review, normalized_diff, relevant_memory, missing_items	Collector output after normalization
Rubric	criteria[], weights, hard_fail_conditions, approval_thresholds	Derived from task intent and plan
CriterionScore	criterion_id, score, confidence, rationale, evidence_refs	Per-item result

EvaluationResult	decision, aggregate_score, blockers, warnings, rationale, next_action	Final output for LangGraph
FeedbackPacket	rejected_criteria, missing_evidence, suggested_rework, escalation_reason	Used by Coding/Recovery agents

Example decision shape:

```
{
  task_id: 'feature-123',
  decision: 'reject',
  aggregate_score: 0.71,
  blockers: ['semantic_mismatch', 'missing_acceptance_case'],
  next_action: 'return_to_coding_with_feedback'
}
```

The contracts should support versioning. Future revisions may change thresholds or rubric semantics without breaking stored audit history.

Important

If a field is missing, model it explicitly as missing rather than fabricating a placeholder value. Missing evidence is itself a signal.

7. State Management

Evaluation state should be mostly ephemeral during a single run and selectively persisted after the decision is made. The coordinator owns the live state while the audit store owns the durable record.

State Item	Owner	Lifetime	Persisted?
Active evaluation request	Coordinator	Single run	No
Evidence bundle	Evidence collector	Single run	Optional for debug
Rubric and criterion mapping	Rubric builder	Single run and benchmark reuse	Yes, when versioned
Scores and decision	Decision policy	Single run	Yes
Feedback packet	Feedback generator	Single run	Yes if rejected
Historical evaluation patterns	Project memory / benchmark store	Cross-run	Yes

State transitions should be monotonic inside a run: collect evidence, derive rubric, score, decide, persist. Do not mutate earlier outputs after later stages have read them.

Future agents consume the persisted outputs as structured facts. The Recovery Agent uses blockers and missing evidence to choose a retry strategy. The Benchmark Harness uses scores and confidence to compare model versions. Memory stores recurring failure patterns, not raw noise.

Memory rule

The evaluator may read memory, but it should not write broad behavioral memory during the same decision unless the run is complete and approved by policy.

8. Component Specifications

Component	Inputs	Outputs	Failure conditions	Future integration
Coordinator	Task context, prior-stage outputs, policy config	Evaluation request, final decision flow	Missing inputs, invalid stage order	LangGraph node boundary
Evidence Collector	Tests, review, diff, memory refs	Normalized evidence bundle	Unreadable artifacts, missing evidence	Failure Recovery / Benchmarks
Rubric Builder	Task intent, plan, constraints	Criteria with weights and hard fails	Ambiguous task spec, insufficient acceptance criteria	Memory of rubric versions
Score Engine	Rubric + evidence bundle	Per-criterion scores + aggregate	Evidence mismatch, unstable scoring	Benchmark Harness metrics
Decision Policy	Scores, confidence, thresholds	Approve / Reject / Escalate	Threshold conflict, policy violation	Deployment gate
Feedback Generator	Rejected criteria, gaps	Actionable handoff note	Empty or vague feedback	Coding/Recovery agents
Audit Logger	Decision record	Durable audit entry	Storage write failure	Observability stack

Implementation detail: keep scoring pure and stateless. The scorer should not alter the evidence bundle or the rubric. That makes it easier to reproduce and test decisions later.

The decision policy is where hard thresholds live. Separate soft judgment from hard gates so that one policy change does not require rewriting the scoring model.

```
pseudo:
  request = build_request(state)
  evidence = collect_evidence(request)
  rubric = build_rubric(request)
  scores = score(rubric, evidence)
  decision = apply_policy(scores, evidence)
  feedback = build_feedback(decision, scores, evidence)
```

9. Implementation Sequence

Step	Build Item	Why now
Step 1	Define schemas for requests, evidence, rubric, scores, and decisions.	Everything else needs stable contracts first.
Step 2	Implement evidence normalization from test and review artifacts.	Later scoring logic depends on consistent inputs.
Step 3	Implement rubric builder for task intent and plan constraints.	The evaluator needs criteria before it can score.
Step 4	Implement score engine with deterministic scoring rules.	Now there is something to judge.
Step 5	Implement decision policy and thresholds.	This turns scores into an actionable outcome.
Step 6	Implement feedback packet generation.	Rejected runs must produce repairable guidance.
Step 7	Implement audit logging and persistence.	Every decision needs traceability.

Step 8	Integrate the evaluator node into LangGraph.	Only now should it gate downstream actions.

The order matters because each later step depends on typed outputs from the earlier step. If you build the policy before the rubric, you get thresholds without meaning. That is just superstition with a config file.

Build discipline

Do not wire the evaluator into the live loop until the standalone decision path is passing golden tests.

10. Validation Strategy

Validation should cover three layers: unit tests for deterministic logic, integration tests for artifact handling, and scenario tests for semantic correctness.

Test layer	What to verify
Unit tests	Rubric generation, scoring math, threshold decisions, feedback formatting.
Integration tests	Artifact parsing, evidence assembly, audit persistence, LangGraph state handoff.
Golden tests	Known task cases where the expected outcome is approve or reject.
Adversarial tests	Solutions that pass tests but fail intent, incomplete fixes, and vague specs.
Regression tests	Past false approvals and false rejections stored as fixtures.

A good evaluator should be conservative where evidence is weak and precise where the task is clear. Measure both false approvals and false rejections. If the layer rejects everything, it is useless; if it approves everything, it is a liability.

Include a benchmark set of tasks with human-labeled verdicts. The evaluator should be measured against that set whenever policy changes or model versions change.

Verification target

The evaluator must produce the same verdict for the same frozen evidence bundle and policy version.

11. Deliverables

Deliverable	Description
Schemas package	Typed request, evidence, rubric, score, and decision contracts.
Coordinator node	LangGraph-facing evaluation entry point.
Evidence normalization module	Artifact ingestion and normalization logic.
Rubric builder	Task-to-criteria translation implementation.
Scoring engine	Deterministic criterion scoring.
Decision policy	Approval / rejection / escalation rules.

Feedback generator	Actionable handoff messages.
Audit logger	Durable evaluation records.
Test suite	Unit, integration, and golden tests.
Benchmark fixtures	Labeled evaluation cases for regression checking.

Each deliverable should be independently inspectable. If a future engineer cannot open a folder and understand what it contains, the deliverable is not finished.

> **Done means done**
>
> The part is complete only when the evaluator can be run against frozen inputs and produce a reproducible decision record.

12. Acceptance Criteria

ID	Testable acceptance criterion
AC-1	Given a complete evidence bundle and a clearly aligned solution, the evaluator returns APPROVE.
AC-2	Given a solution that passes tests but violates the task intent, the evaluator returns REJECT with a semantic-mismatch blocker.
AC-3	Given missing critical evidence, the evaluator returns REJECT or ESCALATE, never APPROVE.
AC-4	Given the same frozen input and policy version, the evaluator returns the same decision and score.
AC-5	Rejected runs emit structured feedback that names the failed criteria and the missing evidence.
AC-6	Every evaluation produces an audit entry with task ID, version, decision, score, and timestamp.
AC-7	The evaluator node integrates with LangGraph without mutating unrelated state.
AC-8	Golden tests cover at least one approve, one reject, and one escalate scenario.

The acceptance criteria must be checked mechanically. If a human has to interpret whether a criterion passed, rewrite the criterion until a test can assert it.

A passed evaluation should be reproducible from its stored artifacts. That is how later parts will trust it instead of treating it like a mysteriously wise opinion.

13. Common Failure Modes

Failure mode	Impact
Over-relying on tests	A solution can pass tests while still solving the wrong problem.
Vague rubric criteria	Weak criteria produce noisy and inconsistent decisions.
Noisy evidence aggregation	Mixing raw logs with summarized evidence can distort scoring.

Reward hacking	The system learns to optimize for rubric wording instead of actual task completion.
Threshold drift	Policy changes quietly change decision rates without benchmark review.
Latency creep	Expensive evidence gathering makes the gate too slow to use.
Unstructured feedback	The Coding Agent cannot act on vague rejections.

The biggest trap is turning the evaluator into a second chatbot. It should not freestyle its way through quality control. Deterministic evidence in, deterministic judgment out.

Another common mistake is letting the evaluator read too much unrelated context. More context is not always better. It often just creates a larger arena for confusion.

14. Future Integration

The Evaluation Layer becomes the decision source for later parts and adjacent systems.

Future component	Connection
Part 8 Failure Recovery	Uses rejection reasons and blocker categories to choose retry strategy.
Part 9 Repository Intelligence	Uses dependency evidence and call-path context to improve semantic judgment.
Part 10 Code Knowledge Graph	Uses structural relationships to verify impact and intent alignment.
Part 11 Context Compression	Uses evaluation outputs to keep only decision-relevant context.
Part 13 Benchmark Harness	Uses scores and verdicts as measurable quality signals.
Deployment & Human Approval	Uses evaluator approval as a prerequisite, not a replacement, for human signoff.

The evaluator should also feed historical patterns into memory: common semantic misses, recurring false approvals, and rubric weaknesses. That feedback loop is essential for improving the rest of the system.

Do not let future parts silently bypass the evaluator unless there is an explicit emergency path. A system that can ignore its own quality gate will eventually do so at the worst possible time.

> **Integration principle**
>
> The evaluator is a gate, not a decoration.

15. Completion Checklist

Check	Item
[ ]	Schemas are versioned and used by coordinator, scorer, policy, and feedback modules.
[ ]	Evidence normalization handles all expected inputs and rejects malformed artifacts.
[ ]	Rubric generation is deterministic for a frozen task spec

	and plan.
[ ]	Scoring is pure, repeatable, and covered by unit tests.
[ ]	Decision policy enforces hard-fail conditions and thresholds.
[ ]	Feedback packets are structured, actionable, and linked to failed criteria.
[ ]	Audit logging persists every run with reproducible references.
[ ]	Golden test suite covers approve, reject, and escalate scenarios.
[ ]	LangGraph integration is isolated and does not mutate unrelated state.
[ ]	Benchmark metrics are available for approval rate, false approval rate, and latency.

When every checkbox can be verified without interpretation, the part is done. When the evaluator is only 'mostly working,' the rest of the system will eventually discover the missing edge case for you, usually in production, because software enjoys humiliation.